MIPHI · POWERED BY PHISON

Onyx with aiDAPTIV+

A private AI assistant that runs on your own hardware and remembers what it has already read.

User Manual and Setup Guide

01

Why it matters

What Onyx and aiDAPTIV+ do, and what your business gets out of them.

02

Setting it up

Step by step instructions for running Onyx with aiDAPTIV+ on your own hardware.

03

Checking it works

How to confirm on your own system that aiDAPTIV+ is saving time.

Version 1.0 · October 2026 · Prepared by MiPhi

Contents

Why Onyx with aiDAPTIV+ 3

A typical example 3
What Onyx is 3
What aiDAPTIV+ adds 4
What happens when someone asks a question 4
Benefits 4
Test results 5
Where it helps most 5
How the setup is organised 6

Part 1: Before You Begin 7

Setting up Docker Desktop 7
Cloning our aiDAPTIV-Onyx Repository 7

Part 2: What's Already Configured For You 8

Part 3: Connecting Onyx to Your LLM Model 10

Part 4: Getting Started 13

Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working 14

Check the cache folder on disk 14
Confirm a cache hit 15

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit 15

Run 2: Cache Hit (KV Reuse) 15

Why Onyx with aiDAPTIV+

Most teams we speak to would like to use AI on their own documents, but two things hold them back. The first is privacy. The well-known AI tools run in someone else's cloud, so every manual, contract or customer record you ask about has to be sent there first. The second is speed. Ask an AI about a long document and you can wait a minute or more before anything shows up on screen.

Onyx with aiDAPTIV+ was put together to deal with both. This chapter explains how, in plain terms. If you would rather go straight to the installation, skip ahead to the Setup Guide.

A typical example

Say your support lead uploads a 20-page product guide and asks how to reset one of your models. The answer is good, but it takes about a minute and a half to start, because the AI has to read the whole guide before it can reply.

Ten minutes later a colleague asks something else about the same guide. In a normal setup the AI reads all 20 pages again, and it will keep doing that for every new question.

With aiDAPTIV+ the first question takes the same time, but the ones after it start in **under a second**. The AI keeps what it learned from the guide and carries on from there. And since everything runs on your own machine, the guide never leaves the building.

What Onyx is

Onyx is a chat assistant, much like ChatGPT, that you host yourself. It runs on your own hardware with an AI model you choose. People type questions in everyday language and get answers based on your documents rather than the open internet.

Everyday language

People ask questions the way they would ask a colleague. There are no commands to learn.

Questions about your files

Attach a manual, policy or report and ask about it, get a summary, or compare it with another one.

Under your control

It runs on your servers, and your administrators decide which models are available and who can use them.

What aiDAPTIV+ adds

When an AI model reads a document, it builds up a set of working notes about the text. Engineers call these notes the **KV cache**. They are normally kept in the graphics card's memory, which is small and expensive, so they get thrown away when space runs out or the server restarts. The next question then has to start from the beginning.

aiDAPTIV+, made by Phison, keeps those notes on a fast SSD instead. There is far more room there, and nothing gets thrown away. When a new question involves material the AI has already read, it loads its notes and carries on.

> **A SIMPLE WAY TO THINK ABOUT IT**
>
> Picture a librarian who answers questions about a book. Without aiDAPTIV+, they read the whole book again every time someone asks something. With aiDAPTIV+, they keep their notes on the shelf and can answer the next person almost straight away, even if the library closed overnight in between.

What happens when someone asks a question

Your users never need to think about any of this, but if you are curious, this is the sequence:

1
You ask
Someone types a question in Onyx in their browser, with or without a file attached.

→

2
Onyx passes it on
Onyx sends the question and any documents to the AI model on your own hardware.

→

3
aiDAPTIV+ checks its notes
Before reading anything again, aiDAPTIV+ looks on the SSD for notes it already has.

→

4
The answer comes back
The reply starts to appear, and any new notes are saved for next time.

Benefits

Benefit	In practice
Your data stays in-house	The model, your documents and your chat history all stay on your own hardware. Nothing is sent to an outside AI provider.
Faster follow-up answers	Follow-up and repeat questions about the same documents reuse earlier work, so they start much sooner.

Benefit	In practice
Survives restarts	The notes sit on the SSD rather than in temporary memory, so the AI still remembers what it has read after the server restarts.
Better use of existing hardware	Long documents and long conversations no longer have to fit in the graphics card's memory, because the SSD takes the overflow.
Predictable cost	There are no per-question or per-token cloud charges. The only limit on usage is your own hardware.
Already configured	Our repository comes with aiDAPTIV+ set up inside Onyx, so there are no configuration files to edit.
Suits teams	Everyone gets a familiar chat screen, and administrators control which models are available and to whom.

Test results

We measured the following in our lab. Part 5 of this guide shows you how to check the same things on your own system.

0.9 s

Time to the first word of an answer about a document of around 10,000 tokens. Without the saved notes, the AI needed 91 seconds to read it again.

10,272 of 10,274

Tokens restored from the SSD after a full server restart. Only two had to be processed again.

2.4 s

Time to the first word in an Onyx benchmark run that reused 12,752 tokens from the SSD. See Part 6 for the details.

Measured by MiPhi in September 2026 using the gemma-4-26B-A4B model (4-bit) on an Intel Core Ultra 7 PC with integrated graphics, running the aiDAPTIV+ middleware with an NVMe SSD cache. Your results will depend on your hardware, the model and the size of your documents.

WHAT THIS MEANS DAY TO DAY

On the same ordinary PC, a second question about a long document started answering roughly 100 times sooner. In practice, that is the difference between going to make a coffee while you wait and reading the answer as it appears.

Where it helps most

Support and service teams answer questions from product manuals and troubleshooting guides all day.

Legal and compliance teams work with contracts and policies that cannot leave the company.

Engineering and R&D teams handle specifications, design documents and test reports that are too sensitive for a public cloud.

HR and operations teams look after handbooks and procedures that people ask the same questions about again and again.

Regulated industries such as finance, healthcare and the public sector often need data to stay on their own premises.

How the setup is organised

The Setup Guide takes you from a fresh machine to your first answer. It is split into six parts:

Part	What you do	Result
Part 1 **Before You Begin**	Install Docker, download our Onyx package and start the AI model server.	Everything Onyx needs is in place.
Part 2 **What's Already Configured**	Look over the aiDAPTIV+ settings we have set up for you.	Nothing to edit, just a quick check.
Part 3 **Connecting Onyx to Your Model**	Point Onyx at your AI model from the Admin Panel.	Onyx can talk to your model.
Part 4 **Getting Started**	Open the chat, pick your model and ask a question.	Your first answer.
Part 5 **Verifying the Cache**	Look at the SSD cache folder and the server log.	Confirmation that aiDAPTIV+ is saving and reusing its notes.
Part 6 **Benchmark**	Check the time to first token when saved work is reused.	Figures you can share with your team.

When you are ready, turn the page and start with Part 1.

Getting it running

Onyx runs as a set of Docker Compose services and connects to an LLM model served separately by aiDAPTIV middleware. This manual uses our repository, which already includes the aiDAPTIV KV-cache settings integrated to onyx.

Part 1: Before You Begin

Setting up Docker Desktop

If you don't already have Docker installed, get Docker Desktop first; everything else in this manual depends on it.

1. Download Docker Desktop from the official site:

https://www.docker.com/products/docker-desktop/

2. Run the installer and accept the defaults. On Windows, make sure the WSL2 option is selected if prompted, as this is Docker's recommended backend.
3. Restart your machine if the installer asks you to.
4. Open Docker Desktop and wait until it says “Engine running” (a green status in the bottom-left corner) before continuing.
5. Confirm it from a terminal:

   ➔ Open and run this command in a separate new terminal



Cloning our aiDAPTIV-Onyx Repository

Use our aiDAPTIV-Integration-Onyx repository. It already has the aiDAPTIV cache settings built into docker-compose.yml, so you can skip straight to docker compose up -d without editing any files yourself:

https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-onyx.git

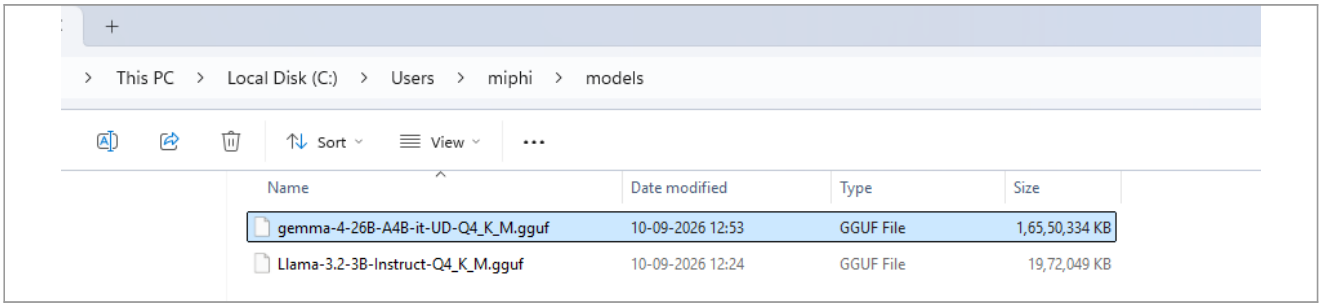
```
git clone https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-onyx.git onyx
cd onyx/deployment/docker_compose
```

From here, follow Onyx's own official Docker deployment guide for everything else (the .env setup, docker compose up -d, and first-time browser setup):

https://docs.onyx.app/deployment/local/docker

The rest of these prerequisites should also be in place before you start Onyx:

6. Your LLM model file (a .gguf file) should be saved on the disk that your aiDAPTIV middleware can reach.
7. The aiDAPTIV middleware is installed as per its own setup guide, and the aiDAPTIV NVMe cache path it uses is configured with the middleware.



Example model folder containing the .gguf file(s) your llama-server will load.

8. Start the LLM model server (via the aiDAPTIV middleware) pointed at that models folder, and confirm it reports a listening address, e.g. http://127.0.0.1:8080. Onyx will connect to this address in Part 3.

> **NOTE**
> Onyx itself does not load the model. It talks to whatever OpenAI-compatible endpoint your aiDAPTIV Middleware exposes. Get that server running and confirmed first.

Part 2: What's Already Configured For You

This section explains the configuration setup of aiDAPTIV

1. Inside api_server's environment: section, the additional parameters required to offload and reuse the KVCache is already present.

```
environment:
  # Auth Settings
  - 'LITELLM_EXTRA_BODY={"cache_prompt": true,"offload_folder_name": "test_2"}'
  - AUTH_TYPE=${AUTH_TYPE:-basic}
```

```yaml
name: onyx

services:
  api_server:
    image: ${ONYX_BACKEND_IMAGE:-onyxdotapp/onyx-backend:${IMAGE_TAG:-latest}}
    build:
      context: ../../backend
      dockerfile: Dockerfile
      # The shipped image; the Dockerfile's default (last) stage is the dev variant
      # with debugging tools (published with a -dev tag suffix).
      target: runtime
    command: >
      /bin/sh -c "alembic upgrade head &&
      echo \"Starting Onyx Api Server\" &&
      uvicorn onyx.main:app --host 0.0.0.0 --port 8080"
    # Check env.template and copy to .env for env vars
    env_file:
      - path: .env
        required: false
    depends_on:
      relational_db:
        condition: service_started
      opensearch:
        condition: service_started
      cache:
        condition: service_started
      inference_model_server:
        condition: service_started
      minio:
        condition: service_started
        required: false
    restart: unless-stopped
    # DEV: To expose ports, either:
    # 1. Use docker-compose.dev.yml: docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --wait
    # 2. Uncomment the ports below
    # ports:
    #   - "8080:8080"
    environment:
      # Auth Settings

  # Auth Settings
    - 'LITELLM_EXTRA_BODY={"cache_prompt": true,"offload_folder_name": "test_2"}'
    - AUTH_TYPE=${AUTH_TYPE:-basic}
    - FILE_STORE_BACKEND=${FILE_STORE_BACKEND:-s3}
    - POSTGRES_HOST=${POSTGRES_HOST:-relational_db}
    - OPENSEARCH_HOST=${OPENSEARCH_HOST:-opensearch}
    - OPENSEARCH_ADMIN_PASSWORD=${OPENSEARCH_ADMIN_PASSWORD:-StrongPassword123!}
    - REDIS_HOST=${REDIS_HOST:-cache}
```

The api_server service block in docker-compose.yml. Note the environment: section near the bottom, which already contains the aiDAPTIV cache line.

What the two keys do:

Key	What it does
`cache_prompt`	Tells the llama-server to save the KV cache generated during prefill instead of discarding it once the reply is sent, and to check for a matching cache before recomputing on the next request.
`offload_folder_name`	The subfolder name, under your aiDAPTIV NVMe cache root, where this model's KV-cache files are written (in this example, “test_2”). You will see this exact name again in Part 5 when you go looking for the cache files on disk.

2. If you ever need a different offload_folder_name (e.g. to keep separate caches per model or benchmark), edit that one line and recreate just that service. Nothing else needs to change:

```
docker compose up -d --force-recreate api_server
```

NOTE

Because this fork already ships with the setting in place, most users can skip straight from Part 1's docker compose up -d to Part 3. This Part is here for reference and verification, not as a required editing step.

Troubleshooting Step

If onyx-nginx-1 container fails to start run the below commands in your powershell to fix it

#First Command :

```
$path = "C:\Users\aidap\onyx\deployment\data\nginx\run-nginx.sh"
$bytes = [System.IO.File]::ReadAllBytes($path)
$text = [System.Text.Encoding]::UTF8.GetString($bytes)
$text = $text -replace "`r`n", "`n"
$text = $text -replace "`r", "`n"
[System.IO.File]::WriteAllText(
   $path,
   $text,
   (New-Object System.Text.UTF8Encoding($false))
)
```

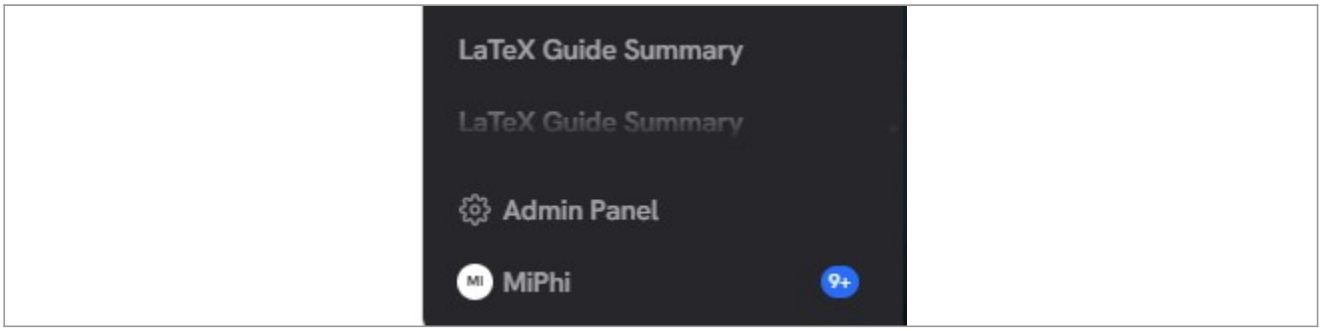
#Second Command :

```
$bytes = [System.IO.File]::ReadAllBytes("C:\Users\aidap\onyx\deployment\data\nginx\run-nginx.sh")
(($bytes | Where-Object { $_ -eq 13 }).Count)
```

Part 3: Connecting Onyx to Your LLM Model

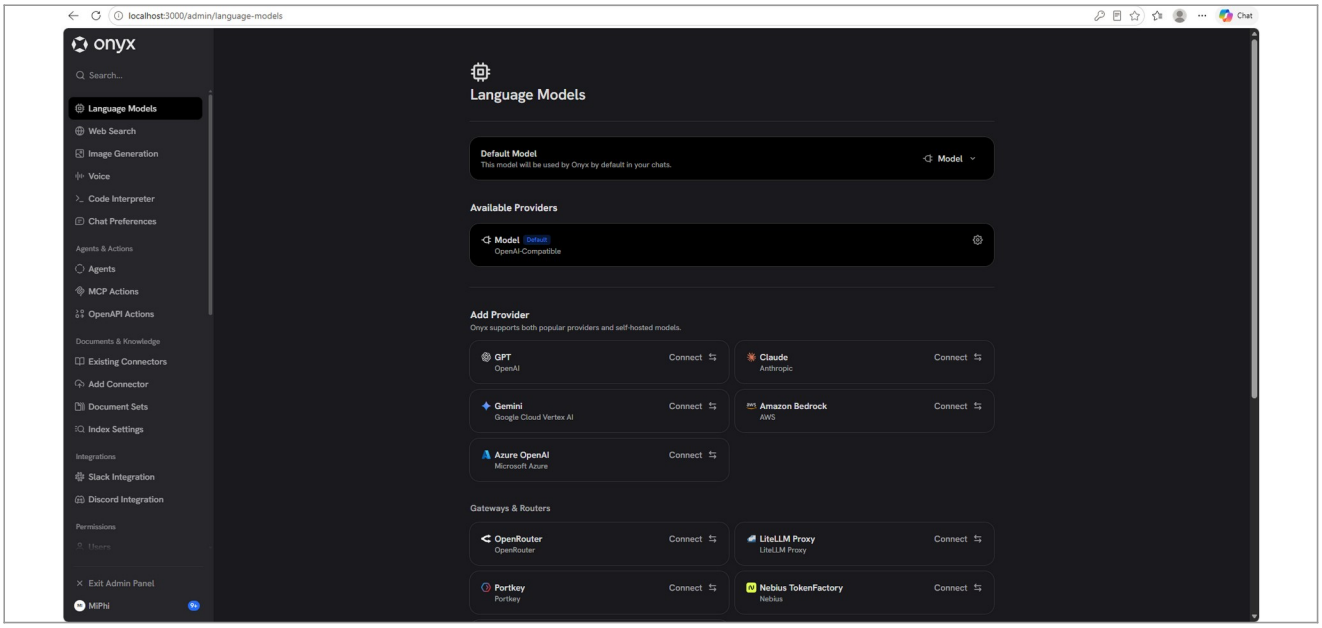
With the model server running (Part 1) and the cache setting in place (Part 2), point Onyx at the model.

1. Go to the URL http://localhost:3000 and Open Onyx in your browser and, from the left sidebar, click Admin Panel. The first user created in Onyx has the Admin Privilege.



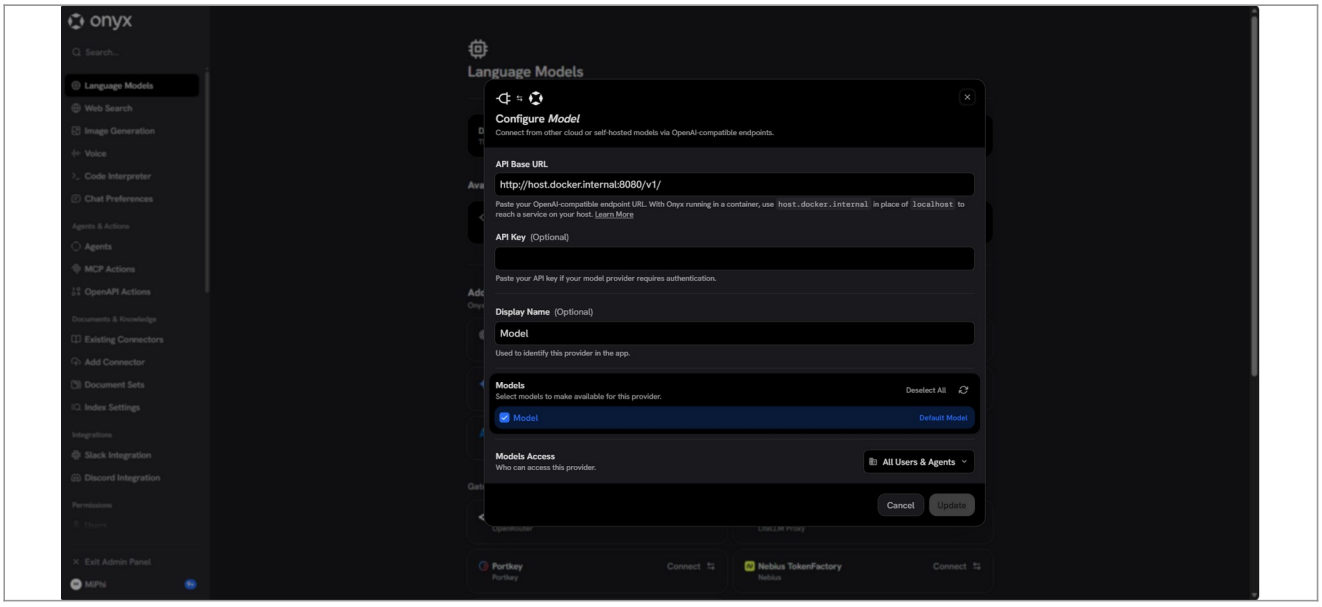
Admin Panel entry point at the bottom of the left sidebar.

2. In the Admin Panel, click Language Models.



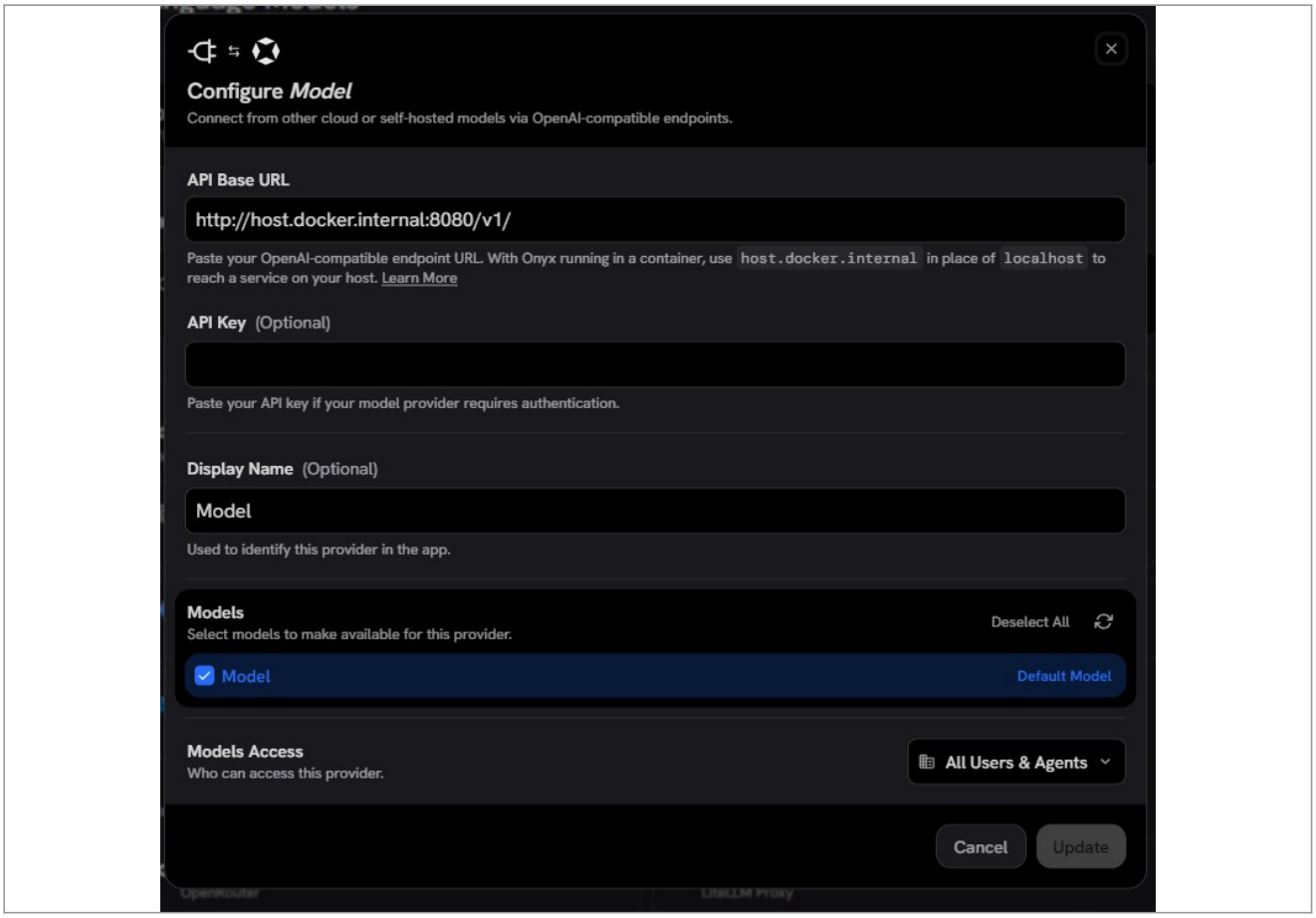
Admin → Language Models. The default 'Model' provider (OpenAI-Compatible) is listed under Available Providers. Click its gear icon to configure it, or Connect under Add Provider to add a new one.

3. In the Configure Model dialog, fill in the fields as follows:



Configure Model dialog.

Field	What to enter
API Base URL	http://host.docker.internal:8080/v1/. Use host.docker.internal, not localhost or 127.0.0.1. Onyx's containers run on their own network and cannot see your host machine's localhost; Docker's host.docker.internal name resolves to it instead.
API Key	Leave blank unless your llama-server is configured to require one.
Display Name	Optional. A label to identify this provider in the app, e.g. “Model.”
Models	Check the box next to your model so it is selectable, and confirm it is marked as the Default Model.
Models Access	Choose who can use it, e.g. All Users & Agents.

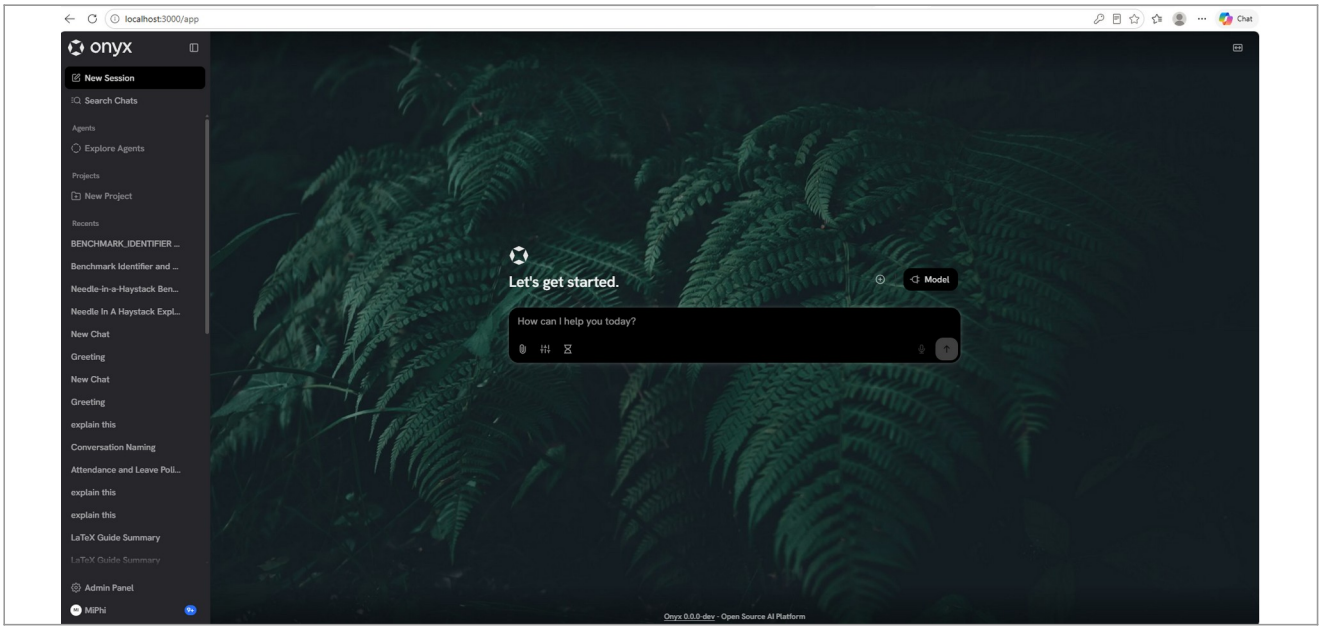


Configure Model dialog, close-up: API Base URL, Models list with the model checked as Default Model, and Models Access.

4. Click Update to save.

Part 4: Getting Started

1. Open http://localhost:3000/app in your browser.



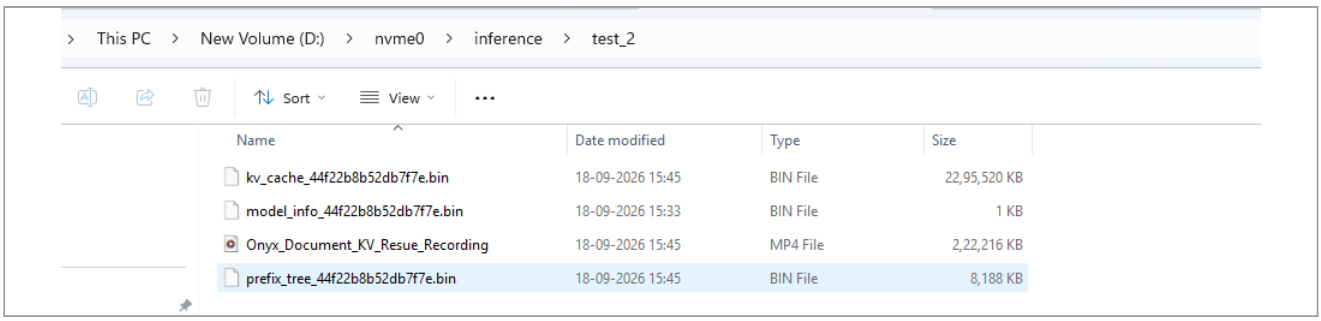
The Onyx 'Let's get started' landing screen.

2. Confirm the Model selector (top of the message box) shows the model you connected in Part 3.
3. Type your prompt and send it. You can also attach a file first and then ask your question about it.

Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working

Check the cache folder on disk

After sending at least one prompt, browse to your aiDAPTIV NVMe cache root plus the offload_folder_name you set in Part 2 (in this example, test_2). New files confirm the KV cache is actually being written to the aiDAPTIV SSD, not just held in VRAM.



Example cache folder D:\nvme0\inference\test_2, showing kv_cache_*.bin, model_info_*.bin, and prefix_tree_*.bin files written by the llama-server.

Confirm a cache hit

Ask a prompt for a first time, The KVCache of the prompt will be saved in aiDAPTIV.Send the same (or an overlapping) prompt again and check the log for the next request:

```
0.18.674.803 I srv          init: init: chat template, thinking = 1
0.18.674.826 I srv  llama_server: model loaded
0.18.674.829 I srv  llama_server: server is listening on http://127.0.0.1:8080
0.18.674.841 I srv  update_slots: all slots are idle
0.59.450.948 I srv  params_from_: Chat format: peg-gemma4
0.59.490.387 I slot get_availabl: id  3 | task -1 | selected slot by LRU, t_last = -1
0.59.490.564 I slot launch_slot_: id  3 | task 0 | processing task, is_child = 0
0.59.490.705 I slot update_slots: id  3 | task 0 | Hit token cnt (GPU): 0
0.59.848.849 I slot update_slots: id  3 | task 0 | Hit token cnt (DRAM): 0
0.59.848.862 I slot update_slots: id  3 | task 0 | Hit token cnt (CACHE): 10976
[MDW][Info][Runtime] Save 0 tokens to folder 'test_2'
```

Second-run log: Hit token cnt (CACHE): 10976, and 0 new tokens saved.

A large, non-zero Hit token cnt (CACHE) value (10,976 tokens in this example) is direct proof that those tokens were retrieved from the aiDAPTIV NVMe cache instead of being recomputed on the GPU. The closing line, “Save 0 tokens to folder 'test_2'”, confirms nothing new needed to be cached because it was already there. This lower recompute workload is what produces the faster Time To First Token you will see on repeated or overlapping prompts.

> **NOTE**
> If your second run still shows Hit token cnt (CACHE): 0, double-check that: the LITELLM_EXTRA_BODY line from Part 2 was added correctly and the containers were recreated (docker compose down && up -d) afterward; the prompt overlaps meaningfully with the first one; and the offload_folder_name folder has free space on the aiDAPTIV SSD.

Part 6: Benchmark Comparison: TTFT, TPS & Cache Hit

Run 2: Cache Hit (KV Reuse)

From our benchmarking test. A previous run had already built up a 12,752-token cache for this prompt; this run reused it:

```
0.39.096.286 I slot get_availabl: id  3 | task -1 | selected slot by LRU, t_last = -1
0.39.096.481 I slot launch_slot_: id  3 | task 0 | processing task, is_child = 0
0.39.096.600 I slot update_slots: id  3 | task 0 | Hit token cnt (GPU): 0
0.39.441.402 I slot update_slots: id  3 | task 0 | Hit token cnt (DRAM): 0
0.39.441.416 I slot update_slots: id  3 | task 0 | Hit token cnt (CACHE): 12752    HIT CACHE
[MDW][Info][Runtime] Save 0 tokens to folder 'benchmark_13k'
0.39.766.394 I slot update_slots: id  3 | task 0 | saved checkpoint to Cache Storage (n_tokens = 12752)
0.41.636.826 I reasoning-budget: activated, budget=2147483647 tokens
0.53.284.748 I slot print_timing: id  3 | task 0 | n_decoded =    100, tg =   8.49 t/s
0.56.344.656 I slot print_timing: id  3 | task 0 | n_decoded =    127, tg =   8.56 t/s
0.57.563.720 I reasoning-budget: deactivated (natural end)
0.59.377.637 I slot update_slots: id  3 | task 0 | Multi-token MoE expert hit rate (GPU): 72.44%
0.59.377.646 I slot update_slots: id  3 | task 0 | Multi-token MoE expert hit rate (DRAM):  0.00%
0.59.377.648 I slot update_slots: id  3 | task 0 | Single-token MoE expert hit rate (GPU): 96.17%
0.59.377.648 I slot update_slots: id  3 | task 0 | Single-token MoE expert hit rate (DRAM):  0.00%
[MDW][Info][Runtime] Save 184 tokens to folder 'benchmark_13k'
0.59.762.249 I slot print_timing: id  3 | task 0 | prompt eval time =    2413.58 ms /    32 tokens (   75.42 ms per token,
13.26 tokens per second)                                                  TTFT
0.59.762.251 I slot print_timing: id  3 | task 0 |        eval time =   17866.91 ms /   155 tokens (  115.27 ms per token,
8.68 tokens per second)  TPS
0.59.762.253 I slot print_timing: id  3 | task 0 |       total time =   20280.48 ms /   187 tokens
```

Real log from benchmark_13k, cache-hit run. Hit Cache: 12,752 tokens. TTFT (prompt eval time): 2,413.58 ms. TPS (eval/decode speed): 8.68 tokens/sec.

Metric	Run 2, warm (cache hit)
Hit token cnt (CACHE)	12,752 tokens
TTFT (prompt eval time)	2,413.58 ms (≈ 2.41 s)
TPS (decode / eval speed)	8.68 tokens/sec